SPENDWISE PROJECT

Team NO : 20

Team Head : S.PREETHI

(81F8F7383E3A301B2414B94EDCC200D3)

TeamMembers:R.S.SIBIKA

(2D50ECE8EBC652E9EC94B1BCE70E695F)

TeamMembers : C.SRUTHI

(300D3172FE2F13897CF4652909151055)

Team Members : M.YOGASRI

(07DF70DD2210435A4834A727229FC04E)

1. Project Title

SpendWise – A Simple Backend Learning Project using Node.js, Express, MongoDB, and JWT

2. Tools and Technologies Used

Node.js – Backend runtime environment

Express.js – Web framework for building APIs

MongoDB – NoSQL database for storing data

Mongoose – MongoDB object modeling tool

JWT (JSON Web Token) – Authentication and security

bcrypt – Password hashing

CORS – Cross-origin resource sharing

dotenv – Environment variable management

Visual Studio Code – Code editor used for development

Postman / Thunder Client – API testing

3. Project Overview

SpendWise is a backend application designed to help beginner developers understand how to build secure REST APIs using Node.js and MongoDB.

The project demonstrates:

User authentication using JWT

CRUD operations

Database integration

Protected API routes

It is designed as a learning project for students and beginner developers.

4. Objectives of the Project

Implement secure user authentication

Perform CRUD operations

Connect backend with MongoDB database

Demonstrate JWT protected routes

Provide a simple backend architecture for learning.

5. This project is useful for:

Target Users

Students learning backend development

Beginner Node.js developers

Freshers preparing for interviews

Developers learning REST APIs

Anyone learning MongoDB and JWT authentication

6. System Requirements

Software Requirements

Node.js

MongoDB

Visual Studio Code

Postman or Thunder Client

Knowledge Requirements

JavaScript (ES6)

Node.js and Express basics

MongoDB and Mongoose

REST API concepts

JWT authentication basics

Git basics.

7. Project Architecture

Backend Architecture

The application follows a layered architecture.

Controllers

Handle the business logic of the application.

Examples:

User registration

Login authentication

Creating transactions

Models

Defines database structure using Mongoose schemas.

Example Models:

User Model

Transaction Model

Routes

Define API endpoints for the application.

Examples:

/api/auth/register

/api/auth/login

/api/transactions

Middleware

Used for security and request validation.

Example:

JWT verification middleware

Configuration

Handles:

Database connection

Environment variables.

8. Database Architecture

The application uses MongoDB as the database.

Users Collection

Stores user account details.

Fields:

_id

name

email

password (hashed)

createdAt

Transactions Collection

Stores transaction records.

Fields:

_id

userId

title

amount

createdAt

Relationship

One User → Many Transactions.

9. Features of SpendWise

User Registration

User Login

JWT Authentication

Secure Password Hashing

Create Transaction

Read Transactions

Update Transaction

Delete Transaction

Protected API Routes.

10. Project Folder Structure

Copy code

```
SpendWise
|
├── config
| └── db.js
|
├── controllers
| ├── authController.js
| └── transactionController.js
|
├── middleware
| └── authMiddleware.js
|
├── models
| ├── User.js
| └── Transaction.js
|
├── routes
```

```
│ ├── authRoutes.js
│ └── transactionRoutes.js
│
├── server.js
├── package.json
└── .env.
```

11. Project Setup and Installation

Step 1: Install Required Tools

Install:

Node.js

MongoDB

Visual Studio Code

Step 2: Initialize Project

Bash

Copy code

npm init

Step 3: Install Dependencies

Bash

Copy code

npm install express mongoose jsonwebtoken bcrypt cors dotenv

Step 4: Run the Server

Bash

Copy code

npm run dev

Step 5: Test APIs

Use:

Postman

Thunder Client

Test API at:

Copy code

http://localhost:5000

12. Roles and Responsibilities

Developer

Write backend code

Create APIs

Test APIs

Maintain database

Backend System

Authenticate users

Manage data storage

Process requests

Protect private routes.

13. Conclusion

SpendWise is a beginner-friendly backend project that demonstrates how to build secure REST APIs using Node.js, Express, MongoDB, and JWT authentication. The project helps students understand authentication, database operations, and API architecture used in real-world applications.